American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Final-Term Project Report

Summer 2025-2026

Section: B

Group: 02

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: NILOY PAUL ID: 23-51773-2	Data preprocessing and corpus preparation. Loaded and class-balanced both corpora, built the duplicate-content hashing and the group-aware 72/8/20 split, ran the duplicate and leakage audit, and implemented the classical normalisation pipeline and the 128-token tokenisation diagnostics.
2.	Name: ARNOB SARKER SUPTA ID: 23-52080-2	BERT experiments. Ran the bert-base-uncased hyperparameter grid over learning rate, batch size and weight decay on both datasets, selected the best configuration on validation weighted F1, and produced the BERT confusion matrices and per-document prediction arrays.
3.	Name: NAZMUS SAKIB ID: 23-52638-2	BERT variant experiments and analysis. Ran the DeBERTa grid on both datasets, carried out the surface and content analysis, the tokenisation and label-free checks, the paired significance testing, and the figures.
4.	Name: AFNAN UR RAYAN ID: 23-51992-2	Ensemble, evaluation and reporting. Built the validation-weighted soft-vote ensemble and its degenerate-weight check, ran the classical baselines and the cross-dataset transfer evaluation, compiled the result tables, and wrote the report.

Detecting AI-Generated Text with BERT, DeBERTa and a Soft-Vote Ensemble

Niloy Paul, Arnob Sarker Supta, Nazmus Sakib and Afnan Ur Rayan
American International University-Bangladesh
Dhaka, Bangladesh
{23-51773-2, 23-52080-2, 23-52638-2, 23-51992-2}@student.aiub.edu

1. Introduction

This project builds a system that decides whether a piece of text was written by a person or produced by a language model. We treat it as a binary classification problem and study it on two public datasets. The first is DAIGT V2, which holds 44,868 argumentative student essays, part written by students and part generated by several 2023-era models. The second is HC3, which holds 85,449 question-and-answer pairs contrasting human answers with answers from ChatGPT. The two are useful together because they differ in almost every way that matters. DAIGT V2 has many generators, one genre and long documents, while HC3 has a single generator, five source domains and short ones.

We balanced both datasets by cutting the larger class down to the size of the smaller one, which leaves 34,994 rows for DAIGT V2 and 53,806 for HC3. We then split each dataset once into training, validation and test parts of 72, 8 and 20 per cent, and every model in this report uses that same split. The split is group aware, meaning that documents with identical text after normalisation are kept together on one side of the boundary. This matters for HC3, where 7.16 per cent of rows are duplicates. Measured directly, our split puts none of the 10,732 HC3 test documents into training, while an ordinary random split of the same data puts 570 of them there. Without the group-aware split, part of the score would be measuring memorisation rather than detection.

Table 1 summarises both datasets as published and after every preprocessing step, so the sizes every later table depends on are stated in one place. Sections 2 through 6 give the model details, grids and reasoning; this section states only the headline result.

The short version of the result is that DeBERTa is the best single model on both datasets, reaching 0.9917 weighted F1 on DAIGT V2 and 0.9972 on HC3. The ensemble does not reliably beat it. We also found something less expected, which is that the two datasets are not equally hard for the same reasons, and Section 6 explains what we mean by that.

Table 1. The two datasets as published and after preprocessing. Duplicate rows are exact matches after whitespace and case normalisation. Leakage is the share of test documents whose text also appears in training or validation. Token counts use the bert-base-uncased tokenizer on a 2,000-row sample per dataset.

	DAIGT V2	HC3
As published		
Total documents	44,868	85,449
Human-written	27,371	58,546
Machine-generated	17,497	26,903
Duplicate rows	6 (0.01%)	6,118 (7.16%)
Collection artefact rows	none found	483
After class balancing		
Total documents	34,994	53,806
Per class	17,497	26,903
After splitting (72 / 8 / 20)		
Training	25,196	38,785
Validation	2,800	4,289
Test	6,998	10,732
Test leakage, group-aware split	0 (0.00%)	0 (0.00%)
Test leakage, ordinary random split	0 (0.00%)	570 (5.30%)
Document length		
Median words per document	346	140
Median words, human	382	83
Median words, machine	328	174
Median tokens per document	415	165
Documents longer than 128 tokens	99.7%	61.1%
Median share read at 128 tokens	30.8%	77.6%

2. Midterm Result Analysis

The midterm project compared three classical models on the same two datasets, each under two text representations, bag-of-words and TF-IDF. All six model and representation combinations were run on the full balanced corpora, using the same split the transformers use, so every number in this report comes from the same data. Table 2 gives the results.

Table 2. Classical models on the full balanced corpora, both text representations, evaluated on the same test split every other model in this report uses. Acc = Accuracy, Prec = Precision, Rec = Recall, F1 = F1 Score.

Model	Representation	Dataset 1 (DAIGT V2)				Dataset 2 (HC3)			
		Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Naive Bayes	BoW	0.9591	0.9603	0.9591	0.9591	0.8713	0.8715	0.8713	0.8713
	TF-IDF	0.9577	0.9578	0.9577	0.9577	0.8672	0.8698	0.8672	0.8670
Logistic Regression	BoW	0.9893	0.9893	0.9893	0.9893	0.9551	0.9551	0.9551	0.9551
	TF-IDF	0.9857	0.9858	0.9857	0.9857	0.9365	0.9365	0.9365	0.9365
Support Vector Machine	BoW	0.9870	0.9870	0.9870	0.9870	0.9475	0.9476	0.9475	0.9475
	TF-IDF	0.9910	0.9911	0.9910	0.9910	0.9449	0.9452	0.9449	0.9449

Best-Average-Worst Model Identify:

Table 3. Best, average, and worst-performing classical model configurations on DAIGT V2 and HC3 based on test-set performance.

Dataset	Performance	Model	Representation	Acc	Prec	Rec	F1
Dataset 1 (DAIGT V2)	Best	Support Vector Machine	TF-IDF	0.9591	0.9603	0.9591	0.9591
	Average	Logistic Regression	BoW	0.9577	0.9578	0.9577	0.9577
	Worst	Naive Bayes	TF-IDF	0.9893	0.9893	0.9893	0.9893
Dataset 2 (HC3)	Best	Logistic Regression	BoW	0.9857	0.9858	0.9857	0.9857
	Average	Support Vector Machine	BoW	0.9870	0.9870	0.9870	0.9870
	Worst	Naive Bayes	TF-IDF	0.9910	0.9911	0.9910	0.9910

The clearest pattern is that the two datasets are not equally difficult. On DAIGT V2 every classical model does well, and the best of them, support vector machine with TF-IDF, reaches 0.9910 F1. On HC3 the same models are noticeably weaker, with the best, logistic regression with BoW, reaching only 0.9551. That is a gap of about 4 points of F1 between the two datasets for the same family of model, so the difficulty belongs to the data rather than to the method.

The choice of representation also matters much more on HC3 than on DAIGT V2. On HC3, moving logistic regression from TF-IDF to bag-of-words changes F1 from 0.9365 to 0.9551, a gain of about two points. On DAIGT V2 the same change moves it from 0.9857 to 0.9893, less than half a point. Bag-of-words wins for five of the six model and dataset combinations, the exception being the support vector machine on DAIGT V2, where TF-IDF is better. We read this as a sign that raw counts of very common tokens carry real signal on HC3, and that TF-IDF weighting, which is designed to discount common tokens, throws part of that signal away.

Naive Bayes is the weakest model on both datasets and by a wide margin on HC3, which is expected given that it treats every word as independent. Logistic regression and the support vector machine are close to each other everywhere.

These results set the target for the transformers. To be worth their extra cost, a fine-tuned model must beat 0.9910 on DAIGT V2 and 0.9551 on HC3.

3 Selecting the Best BERT model

We chose BERT as the base transformer for three practical reasons.

The first is that the model the course introduced, so the comparison is meaningful against what we already knew.

The second is that the model is small enough at 110 million parameters to fine-tune repeatedly on one consumer GPU, which we needed because the grid has eight settings per dataset.

The third is that it is the most widely reported baseline for this task, so our numbers can be compared with published work.

We used bert-base-uncased with a maximum sequence length of 128 tokens, at most five epochs with early stopping after two epochs without improvement, a warmup ratio of 0.1, dropout of 0.1 and the AdamW optimiser. The grid varied learning rate over 0.00002 and 0.00003, batch size over 16 and 32, and weight decay over 0.01 and 0.1, giving eight runs per dataset. Table 4 lists every run. We picked the winner on the validation split, never on the test split, because choosing on test would mean reporting a number we had already optimised against.

Table 4. BERT fine-tuning experiments only. All eight configurations of the sweep, full balanced corpora, evaluated on the test split.

Learning Rate	Batch Size	Weight Decay	Dataset 1 (DAIGT V2)				Dataset 2 (HC3)			
			Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
0.00002	16	0.01	0.9920	0.9920	0.9920	0.9920	0.9910	0.9911	0.9910	0.9910
0.00003	16	0.01	0.9954	0.9954	0.9954	0.9954	0.9940	0.9941	0.9940	0.9940
0.00002	32	0.01	0.9936	0.9936	0.9936	0.9936	0.9944	0.9944	0.9944	0.9944
0.00003	32	0.01	0.9919	0.9919	0.9919	0.9919	0.9929	0.9930	0.9929	0.9929
0.00002	16	0.1	0.9869	0.9870	0.9869	0.9869	0.9916	0.9917	0.9916	0.9916
0.00003	16	0.1	0.9901	0.9902	0.9901	0.9901	0.9911	0.9912	0.9911	0.9911
0.00002	32	0.1	0.9924	0.9924	0.9924	0.9924	0.9945	0.9945	0.9945	0.9945
0.00003	32	0.1	0.9916	0.9916	0.9916	0.9916	0.9933	0.9933	0.9933	0.9933

On DAIGT V2 the best BERT setting is learning rate 3e-05, batch size 32, weight decay 0.1. It reaches 0.9916 weighted F1 on the test split, having scored 0.9957 on validation, and it trained for 5 epochs in 9.7 minutes. Its test confusion matrix is [[3475, 44], [15, 3464]], so of 6,998 documents it misclassifies 59, split roughly evenly between the two error directions.

On HC3 the best setting is learning rate 2e-05, batch size 16, weight decay 0.1, reaching 0.9916 on test after 0.9939 on validation, in 11.5 minutes. Its confusion matrix is [[5304, 73], [17, 5338]], which is 90 errors out of 10,732.

Best-Average-Worst BERT Model Identify:

Table 5. Best, average, and worst-performing BERT hyperparameter configurations on DAIGT V2 and HC3 based on test-set performance.

Dataset	Performance	Learning Rate	Batch Size	Weight Decay	Acc	Prec	Rec	F1
Dataset 1 (DAIGT V2)	Best	0.00003	16	0.01	0.9954	0.9954	0.9954	0.9954
	Average	0.00003	32	0.1	0.9916	0.9916	0.9916	0.9916
	Worst	0.00002	16	0.1	0.9869	0.9870	0.9869	0.9869
Dataset 2 (HC3)	Best	0.00002	32	0.1	0.9945	0.9945	0.9945	0.9945
	Average	0.00003	32	0.01	0.9929	0.9930	0.9929	0.9929
	Worst	0.00002	16	0.01	0.9910	0.9911	0.9910	0.9910

Table 5 above is the complete picture, all three BERT configurations on both datasets, and it supports a best/average/worst reading. On DAIGT V2 the best of the eight is learning rate 3e-05, batch size 16, weight decay 0.01 at 0.9954 F1, the worst is learning rate 2e-05, batch size 16, weight decay 0.1 at 0.9869 F1, the average is learning rate 3e-05, batch size 32, weight decay 0.1 at 0.9916 F1, and the mean across all eight is 0.9917 F1; the spread from worst to best is only 0.0085 F1. On HC3 the best is learning rate 2e-05, batch size 32, weight decay 0.1 at 0.9945 F1, the worst is learning rate 2e-05, batch size 16, weight decay 0.01 at 0.9910 F1, the average is learning rate 3e-05, batch size 32, weight decay 0.01 at 0.9929 F1, and the mean 0.9929 F1, the spread from worst to best is only 0.0035 F1. In other words, once the model and

the data are fixed, the hyperparameters in this grid make very little difference. We select the best-on-validation configuration because the specification requires a single deployed model, not because the gap between best and worst is large enough to call meaningful.

4 Selecting the Best BERT variant model — DeBERTa

For the BERT variant we chose DeBERTa, specifically the microsoft/deberta-v3-base checkpoint. Four things made it the right choice.

The first is benchmark standing. DeBERTa-v3-base is the strongest encoder of its size at the time we selected it, ahead of both BERT-base and RoBERTa-base across the GLUE suite of language-understanding tasks, and the DeBERTa family was the first to pass the human baseline on SuperGLUE. Starting from a stronger pretrained model is the simplest way to get a stronger fine-tuned one, and it costs us nothing extra since the two are close in size.

The second is its attention mechanism, which keeps the representation of a word and the representation of its position separate rather than adding them together. That suits a task where the way text is written may matter as much as which words appear.

The third is how it was pretrained. DeBERTa-v3 is trained to spot tokens that have been replaced by a small generator model, rather than to fill in masked blanks. This gives it a learning signal on every token instead of only the masked ones, so it reaches good accuracy after fewer epochs of fine-tuning, which is what our GPU budget allowed.

The fourth is its tokenizer, and this is the one that turned out to matter most for our data. DeBERTa treats a space before a punctuation mark as part of the token, whereas the BERT tokenizer discards that distinction entirely. We verified this directly rather than assuming it. For the strings "the answer is simple." and "the answer is simple." BERT produces identical token identifiers on three of three test pairs, while DeBERTa distinguishes all three. The two models therefore see genuinely different text, and Section 6 shows why that matters on HC3.

One honest qualification belongs with the first reason. A model that leads a benchmark suite does not automatically lead on a new task, and our own results show exactly that. DeBERTa is far ahead on HC3 but level with BERT on DAIGT V2. Benchmark standing was a sound reason to try it first. It was not a guarantee, and we would not present it as one.

We trained DeBERTa with the same grid, the same split and the same harness as BERT, so any difference between the two is a difference between the models and not between two training setups. Table 6 lists all eight DeBERTa runs per dataset.

Table 6. DeBERTa (BERT variant) fine-tuning experiments only. All eight configurations of the sweep, full balanced corpora, evaluated on the test split.

Learning Rate	Batch Size	Weight Decay	Dataset 1 (DAIGT V2)				Dataset 2 (HC3)			
			Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
0.00002	16	0.01	0.9953	0.9953	0.9953	0.9953	0.9937	0.9937	0.9937	0.9937
0.00003	16	0.01	0.9917	0.9917	0.9917	0.9917	0.9980	0.9980	0.9980	0.9980
0.00002	32	0.01	0.9930	0.9930	0.9930	0.9930	0.9953	0.9954	0.9953	0.9953
0.00003	32	0.01	0.9920	0.9921	0.9920	0.9920	0.9964	0.9964	0.9964	0.9964
0.00002	16	0.1	0.9940	0.9940	0.9940	0.9940	0.9952	0.9952	0.9952	0.9952
0.00003	16	0.1	0.9940	0.9940	0.9940	0.9940	0.9972	0.9972	0.9972	0.9972
0.00002	32	0.1	0.9934	0.9934	0.9934	0.9934	0.9964	0.9964	0.9964	0.9964
0.00003	32	0.1	0.9949	0.9949	0.9949	0.9949	0.9966	0.9967	0.9966	0.9966

On DAIGT V2 the best DeBERTa setting is learning rate 3e-05, batch size 16, weight decay 0.01, reaching 0.9917 weighted F1 on test after 0.9943 on validation, in 12.2 minutes. Its confusion matrix is [[3491, 28], [30, 3449]].

On HC3 the best setting is learning rate 3e-05, batch size 16, weight decay 0.1, and this is where DeBERTa clearly separates from BERT. It reaches 0.9972 on test after 0.9979 on validation, with confusion matrix [[5360, 17], [13, 5342]], which is only 30 errors out of 10,732. BERT made 90 errors on the same documents, so DeBERTa removes two thirds of them.

Best-Average-Worst BERT variation DeBERTa Model Identify:

Table 7. Best, average, and worst-performing DeBERTa hyperparameter configurations on DAIGT V2 and HC3 based on test-set performance.

Dataset	Performance	Learning Rate	Batch Size	Weight Decay	Acc	Prec	Rec	F1
Dataset 1 (DAIGT V2)	Best	0.00002	16	0.01	0.9953	0.9953	0.9953	0.9953
	Average	0.00002	32	0.1	0.9934	0.9934	0.9934	0.9934
	Worst	0.00003	16	0.01	0.9917	0.9917	0.9917	0.9917
Dataset 2 (HC3)	Best	0.00003	16	0.01	0.9980	0.9980	0.9980	0.9980
	Average	0.00002	32	0.1	0.9964	0.9964	0.9964	0.9964
	Worst	0.00002	16	0.01	0.9937	0.9937	0.9937	0.9937

Table 7 above is the complete three configuration DeBERTa grid on both datasets. On DAIGT V2 the best of the eight is learning rate 2e-05, batch size 16, weight decay 0.01 at 0.9953 F1, the worst is learning rate 3e-05, batch size 16, weight decay 0.01 at 0.9917 F1, the average is learning rate 2e-05, batch size 32, weight decay 0.1 at 0.9934 F1, and the mean 0.9935 F1, the spread from worst to best is only 0.0036 F1. On HC3 the best is learning rate 3e-05, batch size 16, weight decay 0.01 at 0.9980, the worst is learning rate 2e-05, batch size 16, weight decay 0.01 at 0.9937 F1, the average is learning rate 2e-05, batch size 32, weight decay 0.1 at 0.9964 F1, and the mean 0.9961 F1, the spread from worst to best is only 0.0043 F1. As with BERT, the spread is small next to the gap between models, which is why we select on validation F1 rather than treating the best-vs-worst gap inside one model as meaningful.

The five fixed settings the instructor specified, five epochs with early stopping, a warmup ratio of 0.1, a maximum sequence length of 128 tokens, dropout of 0.1, and the AdamW optimiser, are not an alternative to our eight-point grid; they are the fixed part of the harness the grid runs inside. Every one of the eight DeBERTa runs, and every one of the eight BERT runs in Section 3, used all five exactly as specified. The eight-point grid is a separate axis, learning rate, batch size and weight decay, layered on top of those five fixed settings because the specification also requires a hyperparameter sweep, and five fixed values alone would not produce one.

The comparison between the two models is not the same on the two datasets, and that is the most useful thing this section shows. On DAIGT V2 the two are level, 0.9916 against 0.9917, a difference of one ten-thousandth that we would not call a difference at all. On HC3 the gap is real and large. Any conclusion of the form "DeBERTa is better than BERT for detecting AI text" would be true for one of our datasets and false for the other.

5 Ensemble Model

Our ensemble model combines bert-base-uncased and microsoft/deberta-v3-base (BERT variant), each at its validation-selected best configuration from Sections 3 and 4, by averaging the class probabilities they produce, which is usually called soft voting. Rather than fixing an equal average, we searched a weight w between 0 and 1 in steps of 0.05, forming w times the BERT probabilities plus one minus w times the DeBERTa probabilities. We chose on the validation split and then applied it once to the test split. Choosing the weight on test would have meant tuning on the data we report. Table 8 gives the two ensemble rows, one per dataset, with the weight and the resulting metrics.

We chose a soft-vote ensemble combining BERT and DeBERTa for three practical reasons: complementary predictions, reduced dependence on a single model, and validation-based weight selection.

1. Complementary Predictions: Combine BERT and DeBERTa to exploit potentially different prediction patterns and errors.

2. Validation-Based Adaptability: Select the contribution of each model from the validation set instead of assuming equal importance.

3. Robust Model Comparison: Test whether combining two strong transformers can improve over the best individual model, while avoiding test-set tuning.

Table 8. Ensemble model only. The validation-selected mixing weight on BERT and the resulting test-set metrics, one row per dataset.

Dataset	Weight on BERT	Acc	Prec	Rec	F1
DAIGT V2	0.50	0.9936	0.9936	0.9936	0.9936
HC3	0.00	0.9972	0.9972	0.9972	0.9972

On DAIGT V2 the search selected w = 0.50, an equal blend of the two models. The ensemble reaches 0.9936 weighted F1, against 0.9916 for BERT alone and 0.9917 for DeBERTa alone. That looks like a clear gain, and it is the point where it would be easy to overstate the result. We compared the ensemble with DeBERTa on the same test documents using a paired test, which gives p = 0.085 and a 95 per cent interval on the difference in error rate of minus 0.39 to plus 0.01 percentage points. That interval contains zero, so on this test set the improvement cannot be told apart from chance. We report the ensemble as nominally ahead, not as better.

On HC3 the search selected w = 0.00. That means it put no weight at all on BERT, so the ensemble is simply DeBERTa and scores exactly what DeBERTa scores, 0.9972. We report this openly rather than presenting the number as an ensemble result, because a soft-vote ensemble that has discarded one of its two members is not really an ensemble.

The honest summary is that ensembling did not help us. On the dataset where the two models are level, blending them produced a small gain we cannot confirm statistically, and on the dataset where one model is clearly better, the weight search simply picked that model. This is a reasonable outcome rather than a failure. Ensembles help most when their members make different kinds of mistakes, and on HC3 DeBERTa makes so few mistakes that BERT has little to add.

6 Overall Result Analysis

This section shows two required final tables. Table 9 is the full sweep, all eight configurations of BERT and DeBERTa on both datasets plus the ensemble, moved here from the per-model sections so the whole grid can be read in one place. Table 10 below it is the final six-row comparison, one row per model family at its best setting; each classical model name carries its winning representation directly, for example "Naive Bayes [BoW]", instead of a separate Representation column. Each classical model uses the same representation on both datasets, chosen by summed F1 across the two, so a model is never shown at one representation on DAIGT V2 and a different one on HC3. Naive Bayes and Logistic Regression both land on BoW under this rule, unchanged from picking per dataset. The Support Vector Machine does not, per dataset TF-IDF wins on DAIGT V2 and BoW wins on HC3, and summed F1 favours TF-IDF by 0.0014, so the SVM row uses TF-IDF on both datasets, 0.0026 F1 short of BoW's HC3 score rather than at BoW's HC3 best.

Table 9. Final-term fine-tuning experiments, all models. Eight configurations for each of BERT and DeBERTa on each dataset, full balanced corpora, evaluated on the test split. The ENSEMBLE row uses the weight chosen on validation.

Model	Learning Rate	Batch Size	Weight Decay	Dataset 1 (DAIGT V2)				Dataset 2 (HC3)			
				Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
BERT	0.00002	16	0.01	0.9920	0.9920	0.9920	0.9920	0.9910	0.9911	0.9910	0.9910
	0.00003	16	0.01	0.9954	0.9954	0.9954	0.9954	0.9940	0.9941	0.9940	0.9940
	0.00002	32	0.01	0.9936	0.9936	0.9936	0.9936	0.9944	0.9944	0.9944	0.9944
	0.00003	32	0.01	0.9919	0.9919	0.9919	0.9919	0.9929	0.9930	0.9929	0.9929
	0.00002	16	0.1	0.9869	0.9870	0.9869	0.9869	0.9916	0.9917	0.9916	0.9916
	0.00003	16	0.1	0.9901	0.9902	0.9901	0.9901	0.9911	0.9912	0.9911	0.9911
	0.00002	32	0.1	0.9924	0.9924	0.9924	0.9924	0.9945	0.9945	0.9945	0.9945
	0.00003	32	0.1	0.9916	0.9916	0.9916	0.9916	0.9933	0.9933	0.9933	0.9933
DeBERTa (BERT variant)	0.00002	16	0.01	0.9953	0.9953	0.9953	0.9953	0.9937	0.9937	0.9937	0.9937
	0.00003	16	0.01	0.9917	0.9917	0.9917	0.9917	0.9980	0.9980	0.9980	0.9980
	0.00002	32	0.01	0.9930	0.9930	0.9930	0.9930	0.9953	0.9954	0.9953	0.9953
	0.00003	32	0.01	0.9920	0.9921	0.9920	0.9920	0.9964	0.9964	0.9964	0.9964
	0.00002	16	0.1	0.9940	0.9940	0.9940	0.9940	0.9952	0.9952	0.9952	0.9952
	0.00003	16	0.1	0.9940	0.9940	0.9940	0.9940	0.9972	0.9972	0.9972	0.9972
	0.00002	32	0.1	0.9934	0.9934	0.9934	0.9934	0.9964	0.9964	0.9964	0.9964
	0.00003	32	0.1	0.9949	0.9949	0.9949	0.9949	0.9966	0.9967	0.9966	0.9966
ENSEMBLE	(validation-selected weight)			0.9936	0.9936	0.9936	0.9936	0.9972	0.9972	0.9972	0.9972

Across Table 10's six rows, the best on DAIGT V2 is ENSEMBLE at 0.9936 F1, the worst is Naive Bayes [BoW] at 0.9591, mean 0.9860. On HC3 the best is DeBERTa (BERT variant) at 0.9972, the worst is Naive Bayes [BoW] at 0.8713, mean 0.9596. DeBERTa is best on both because disentangled attention and a replaced-token-detection pretraining objective give it a genuine edge on HC3's formatting-carried signal, discussed below, while on DAIGT V2 it merely ties the classical ceiling rather than beating it. Naive Bayes is worst on both for the same independence-assumption reason given in Section 2, and it stays worst even after being allowed its better representation.

Table 10. Final comparison, six rows. Every row comes from the full balanced corpora and the same fixed split. Each classical model name carries the text representation it uses, e.g. "Naive Bayes [BoW]", the same representation on both datasets for that model, chosen by summed F1 across the two. The ENSEMBLE row uses the weight chosen on validation.

Model	Dataset 1 (DAIGT V2)				Dataset 2 (HC3)			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Naive Bayes [BoW]	0.9591	0.9603	0.9591	0.9591	0.8713	0.8715	0.8713	0.8713
Logistic Regression [BoW]	0.9893	0.9893	0.9893	0.9893	0.9551	0.9551	0.9551	0.9551
Support Vector Machine [TF-IDF]	0.9910	0.9911	0.9910	0.9910	0.9449	0.9452	0.9449	0.9449
BERT	0.9916	0.9916	0.9916	0.9916	0.9916	0.9917	0.9916	0.9916
DeBERTa (BERT variant)	0.9917	0.9917	0.9917	0.9917	0.9972	0.9972	0.9972	0.9972
ENSEMBLE	0.9936	0.9936	0.9936	0.9936	0.9972	0.9972	0.9972	0.9972

Reading the table across, three further conclusions hold. The first is that DeBERTa is the best single model on both datasets. The second is that the size of its advantage depends entirely on which dataset you look at. On DAIGT V2 the best classical model reaches 0.9910 and DeBERTa reaches 0.9917, a difference of 0.07 of a percentage point, so the extra cost of a transformer buys almost nothing there. On HC3 the same comparison is 0.9551 against 0.9972, a gap of more than four points, and the transformer is clearly doing something the classical models cannot. The third is that the ensemble adds nothing we can demonstrate.

It is worth being precise about the DAIGT V2 result rather than rounding it away. The classical model reads the whole document while the transformers read only their first 128 tokens, which is about a third of the average essay. When we refitted the classical models on the same 128-token span the transformers see, their error rate rose from 0.90 to 2.10 per cent while BERT stayed at 0.84. So, the apparent tie on DAIGT V2 is not evidence that a bag-of-words model is as good as a transformer. It is evidence that reading the whole essay is worth about as much as being a transformer, which is a different and more useful statement.

We also ran one experiment that is not required by the specification but explains the pattern above, and it is the most interesting thing we found. We built two restricted models. One reads only 47 surface properties of the text, such as how often a space appears before a full stop, how long the document is, and how often capital letters are used. It never sees a single word. The other reads only the words, after punctuation, capitalisation and unusual characters have been stripped out. On HC3 these two score almost identically, with error rates of 3.20 and 3.26 per cent, and a paired test cannot separate them. In other words, on HC3 you can do about as well by looking at how the text is punctuated as by reading what it says. On DAIGT V2 the picture is different, with 7.86 per cent error from surface alone against 0.99 per cent from words alone, so the words carry roughly eight times more of the signal.

This explains why HC3 looked harder for the classical models and easier for DeBERTa. Much of what separates the classes on HC3 is in the formatting, and the classical pipeline deletes formatting during preprocessing while DeBERTa keeps it. It also means a high score on HC3 should be read carefully, because part of it comes from how the dataset was collected rather than from any real difference between human and machine writing.

7 Limitations

Several things limit what our numbers can be taken to mean, and we would rather state them than leave them to be discovered.

1. Both transformers read at most 128 tokens per document. On HC3 that covers about three quarters of the average answer, but on DAIGT V2 it covers only about a third of the average essay. So, the DAIGT V2 transformer results really describe classifying an essay from its opening, and the comparison with the classical models, which read the whole document, is not a fair fight in either direction.
2. Every transformer number comes from a single training seed. We repeated a few settings with different seeds and saw the score move by up to a few thousandths, and one such estimate changed by about a third when we simply reran it, because training on a GPU is not perfectly repeatable. For that reason, we did not base any claim on small differences between runs, and where we compare two models, we compare their predictions on the same documents instead.
3. Both datasets are English, and one of them, HC3, contains output from a single generator collected at a single point in time. Nothing in this report shows that our models would work on text from a newer model, on another language, or on a domain neither dataset covers. We checked the weakest version of this by training on one dataset and testing on the other, and performance falls by between 8 and 20 points of F1, so the models clearly learn things specific to their own dataset.
4. Finally, we did not test what happens when someone actively tries to defeat the detector. We ran some simple text corruption experiments, but we also found that our models confidently label random character strings as human-written, which means a drop in score under corruption cannot be separated from the model simply not coping with unreadable input. We therefore do not report those experiments as evidence of robustness in either direction.

8 Conclusion

We built and compared five families of model for deciding whether text is human-written or machine-generated, on two public datasets, using one fixed split and one training harness so that the comparisons are fair. DeBERTa is the best single model on both, reaching 0.9917 weighted F1 on DAIGT V2 and 0.9972 on HC3, which improves on the best classical model by 0.07 of a percentage point on the first dataset and by more than four points on the second. A soft-vote ensemble of BERT and DeBERTa did not reliably improve on DeBERTa alone, and we report that plainly rather than presenting a difference in the fourth decimal place as a gain.

The result we think is most worth carrying forward is not the leaderboard. It is that a model reading only 47 formatting properties of a document, and no words at all, matches a full bag-of-words model on HC3 while falling far behind it on DAIGT V2. That tells us the two datasets are separable for different reasons, which a single accuracy figure hides completely. Anyone choosing a dataset to evaluate a detector, or reading a reported score, should want to know which of the two situations they are in. The check is cheap, it needs no extra labelling, and it runs in minutes, so we would recommend running it before trusting any headline number on this task.

Detecting Machine-Generated Text, Project Code

1. Data preprocessing code

```
import os

os.environ.setdefault('HF_HOME', '/media/filwel/MLProject1/hf_cache')
os.environ.setdefault('HF_HUB_DISABLE_SYMLINKS', '1')
os.environ.setdefault('PYTORCH_CUDA_ALLOC_CONF', 'expandable_segments:True')
os.environ.setdefault('TOKENIZERS_PARALLELISM', 'false')

import gc
import hashlib
import json
import re
import shutil
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import torch

warnings.filterwarnings('ignore')
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True

PROJECT_DIR = Path('/media/filwel/All/Sakib/Semester 10/ NATURAL LANGUAGE PROCESSING /Project ')
FINAL_DIR = Path('/media/filwel/All/Sakib/Semester 10/ NATURAL LANGUAGE PROCESSING /Final')

PS_DIR = FINAL_DIR / 'experiments' / 'paper_scale'
WORK_DIR = PS_DIR / 'work'
RESULTS_DIR = PS_DIR / 'results'
PROBS_DIR = PS_DIR / 'probs'
MODELS_DIR = PS_DIR / 'models'
CKPT_DIR = Path('/media/filwel/MLProject1/nlp_paper_ckpt')
for d in (WORK_DIR, RESULTS_DIR, PROBS_DIR, MODELS_DIR, CKPT_DIR):
    d.mkdir(parents=True, exist_ok=True)

MAX_LEN = 128
EPOCHS = 5
WARMUP_RATIO = 0.1
PATIENCE = 2
SPLIT_SEED = 42
TRAIN_SEED = 42

MODELS = {'BERT': 'bert-base-uncased', 'DeBERTa': 'microsoft/deberta-v3-base'}
DATASET_NAMES = {'D1': 'DAIGT V2', 'D2': 'HC3'}

print('torch', torch.__version__, '| cuda', torch.cuda.is_available())

from sklearn.model_selection import GroupShuffleSplit


def normalise(t):
    return re.sub(r'\s+', ' ', str(t)).strip()


def content_hash(series):
    return series.map(lambda t: hashlib.md5(normalise(t).lower().encode()).hexdigest())


def balance(df, seed=SPLIT_SEED):
    n = int(df['label'].value_counts().min())
    parts = [df[df['label'] == v].sample(n=n, random_state=seed)
             for v in sorted(df['label'].unique())]
    return pd.concat(parts).sample(frac=1, random_state=seed).reset_index(drop=True)


def load_D1():
    raw = pd.read_csv(PROJECT_DIR / 'daigt.csv')
    df = raw[['text', 'label']].dropna()
    df['label'] = df['label'].astype(int)
    del raw
    gc.collect()
    return balance(df)


def load_D2():
    raw = pd.read_json(PROJECT_DIR / 'hc3.jsonl', lines=True)
    human = raw[['human_answers']].explode('human_answers').rename(
        columns={'human_answers': 'text'})
    human['label'] = 0
    bot = raw[['chatgpt_answers']].explode('chatgpt_answers').rename(
        columns={'chatgpt_answers': 'text'})
```

```
    bot['label'] = 1
    df = pd.concat([human, bot], ignore_index=True).dropna()
    df['text'] = df['text'].astype(str)
    del raw, human, bot
    gc.collect()
    return balance(df)


LOADERS = {'D1': load_D1, 'D2': load_D2}


def group_split(df, seed=SPLIT_SEED):
    groups = df['hash'].values
    gss1 = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=seed)
    tr_full, te = next(gss1.split(df, df['label'], groups))
    sub = df.iloc[tr_full]
    gss2 = GroupShuffleSplit(n_splits=1, test_size=0.1, random_state=seed)
    tr_rel, val_rel = next(gss2.split(sub, sub['label'], sub['hash'].values))
    idx_tr, idx_val = sub.index.values[tr_rel], sub.index.values[val_rel]
    idx_te = df.index.values[te]
    g_tr = set(df.loc[idx_tr, 'hash'])
    g_val = set(df.loc[idx_val, 'hash'])
    g_te = set(df.loc[idx_te, 'hash'])
    assert not (g_tr & g_val) and not (g_tr & g_te) and not (g_val & g_te)
    return idx_tr, idx_val, idx_te


def build_or_load_splits(tag, rebuild=False):
    data_p = WORK_DIR / f'data_{tag}.parquet'
    split_p = WORK_DIR / f'split_{tag}.npz'
    if not rebuild and data_p.exists() and split_p.exists():
        df = pd.read_parquet(data_p)
        sp = np.load(split_p)
        return df, {'train': sp['train'], 'val': sp['val'], 'test': sp['test']}
    df = LOADERS[tag]()
    df['hash'] = content_hash(df['text'])
    idx_tr, idx_val, idx_te = group_split(df)
    df[['text', 'label']].to_parquet(data_p, index=True)
    np.savez(split_p, train=idx_tr, val=idx_val, test=idx_te)
    return df[['text', 'label']], {'train': idx_tr, 'val': idx_val, 'test': idx_te}


DATA, SPLITS = {}, {}
for tag in ('D1', 'D2'):
    DATA[tag], SPLITS[tag] = build_or_load_splits(tag)
    n = {k: len(v) for k, v in SPLITS[tag].items()}
    total = sum(n.values())
    print(f'{tag} {DATASET_NAMES[tag]:9s} total={total:6d}  train={n["train"]}  '
          f'val={n["val"]}  test={n["test"]}')

import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

for pkg in ('punkt', 'punkt_tab', 'stopwords', 'wordnet', 'omw-1.4'):
    nltk.download(pkg, quiet=True)

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))


def preprocess_classical(text):
    text = re.sub(r'[^a-z\s]', ' ', str(text).lower())
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 1]
    return ' '.join(tokens)


demo = DATA['D2']['text'].iloc[0]
print('raw       ', repr(demo[:120]))
print('normalised', repr(normalise(demo)[:120]))
print('classical ', repr(preprocess_classical(demo)[:120]))

from datasets import Dataset
from transformers import AutoTokenizer

_TOKCACHE, _DATACACHE = {}, {}


def get_tokenizer(model_key):
    if model_key not in _TOKCACHE:
        _TOKCACHE[model_key] = AutoTokenizer.from_pretrained(MODELS[model_key])
    return _TOKCACHE[model_key]


def get_tokenized(tag, model_key):
    key = (tag, model_key)
    if key in _DATACACHE:
        return _DATACACHE[key]
```

```
    df, splits = DATA[tag], SPLITS[tag]
    tok = get_tokenizer(model_key)
    parts = {}
    for split, idx in splits.items():
        sub = df.loc[idx]
        ds = Dataset.from_dict({'text': [normalise(t) for t in sub['text']],
                                'labels': [int(v) for v in sub['label']]})
        parts[split] = ds.map(
            lambda b: tok(b['text'], truncation=True, max_length=MAX_LEN),
            batched=True, remove_columns=['text'])
    _DATACACHE[key] = (parts, splits)
    gc.collect()
    return _DATACACHE[key]
```

2. BERT code where the best performance was achieved

```
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_recall_fscore_support)
from transformers import (AutoModelForSequenceClassification,
                          DataCollatorWithPadding, EarlyStoppingCallback,
                          Trainer, TrainingArguments, set_seed)


def weighted_metrics(y, p):
    acc = accuracy_score(y, p)
    pre, rec, f1, _ = precision_recall_fscore_support(
        y, p, average='weighted', zero_division=0)
    return acc, pre, rec, f1


def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc, pre, rec, f1 = weighted_metrics(labels, preds)
    return {'accuracy': acc, 'precision': pre, 'recall': rec, 'f1': f1}


def run_key(tag, model_key, cfg, seed=TRAIN_SEED):
    return (f'full_{tag}_{model_key}_lr{cfg["lr"]:g}_bs{cfg["bs"]}'
            f'_wd{cfg["wd"]:g}_s{seed}')


def train_one(tag, model_key, cfg, seed=TRAIN_SEED, save_model=True):
    learning_rate = cfg['lr']
    batch_size = cfg['bs']
    weight_decay = cfg['wd']
    key = run_key(tag, model_key, cfg, seed)
    run_dir = CKPT_DIR / key
    parts, splits = get_tokenized(tag, model_key)
    tok = get_tokenizer(model_key)

    set_seed(seed)
    model = AutoModelForSequenceClassification.from_pretrained(
        MODELS[model_key], num_labels=2)
    model.config.id2label = {0: 'human', 1: 'ai'}
    model.config.label2id = {'human': 0, 'ai': 1}

    args = TrainingArguments(
        output_dir=str(run_dir),
        learning_rate=learning_rate,
        per_device_train_batch_size=batch_size,
        per_device_eval_batch_size=64,
        weight_decay=weight_decay,
        num_train_epochs=EPOCHS,
        warmup_ratio=WARMUP_RATIO,
        lr_scheduler_type='linear',
        optim='adamw_torch',
        bf16=torch.cuda.is_bf16_supported(),
        eval_strategy='epoch',
        save_strategy='epoch',
        save_total_limit=1,
        load_best_model_at_end=True,
        metric_for_best_model='eval_f1',
        greater_is_better=True,
        logging_steps=200,
        seed=seed,
        data_seed=seed,
        dataloader_num_workers=0,
        report_to='none')

    trainer = Trainer(
        model=model, args=args, train_dataset=parts['train'],
        eval_dataset=parts['val'], data_collator=DataCollatorWithPadding(tok),
        compute_metrics=compute_metrics,
        callbacks=[EarlyStoppingCallback(early_stopping_patience=PATIENCE)])

    t0 = time.time()
    trainer.train()
```

```
    train_secs = time.time() - t0

    out = {'key': key, 'dataset': tag, 'dataset_name': DATASET_NAMES[tag],
           'model': model_key, 'checkpoint': MODELS[model_key],
           'lr': learning_rate, 'batch_size': batch_size,
           'weight_decay': weight_decay, 'seed': seed, 'max_len': MAX_LEN,
           'n_train': len(splits['train']), 'n_val': len(splits['val']),
           'n_test': len(splits['test']), 'train_seconds': round(train_secs, 1),
           'epochs_run': int(trainer.state.epoch or 0)}

    probs = {}
    for split in ('val', 'test'):
        pred = trainer.predict(parts[split])
        raw = pred.predictions[0] if isinstance(pred.predictions, tuple) else pred.predictions
        p = torch.softmax(torch.tensor(raw, dtype=torch.float32), dim=-1).numpy()
        y = np.asarray(pred.label_ids)
        acc, pre, rec, f1 = weighted_metrics(y, p.argmax(1))
        out[split] = {'accuracy': round(acc, 4), 'precision': round(pre, 4),
                      'recall': round(rec, 4), 'f1': round(f1, 4)}
        out[f'{split}_confusion'] = confusion_matrix(y, p.argmax(1)).tolist()
        probs[f'{split}_probs'] = p
        probs[f'{split}_labels'] = y

    np.savez(PROBS_DIR / f'{key}.npz', **probs)
    json.dump(out, open(RESULTS_DIR / f'{key}.json', 'w'), indent=2)

    if save_model:
        mdir = MODELS_DIR / f'{tag}_{model_key}'
        trainer.save_model(str(mdir))
        tok.save_pretrained(str(mdir))
        json.dump(out, open(mdir / 'run_info.json', 'w'), indent=2)

    del trainer, model
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    shutil.rmtree(run_dir, ignore_errors=True)
    print(f'{key}  val_f1={out["val"]["f1"]:.4f}  test_f1={out["test"]["f1"]:.4f}  '
          f'{train_secs / 60:.1f} min')
    return out

BERT_BEST = {
    'D1': {'lr': 3e-5, 'bs': 32, 'wd': 0.1},
    'D2': {'lr': 2e-5, 'bs': 16, 'wd': 0.1},
}

BERT_RESULT, BERT_CFG = {}, {}
for tag in ('D1', 'D2'):
    cfg = BERT_BEST[tag]
    print(f'{tag} {DATASET_NAMES[tag]:9s} BERT  learning_rate={cfg["lr"]:g}  '
          f'batch_size={cfg["bs"]}  weight_decay={cfg["wd"]:g}')
    BERT_CFG[tag] = dict(cfg, key=run_key(tag, 'BERT', cfg))
    BERT_RESULT[tag] = train_one(tag, 'BERT', cfg)
```

3. BERT variant code, DeBERTa, where the best performance was achieved

```
DEBERTA_BEST = {
    'D1': {'lr': 3e-5, 'bs': 16, 'wd': 0.01},
    'D2': {'lr': 3e-5, 'bs': 16, 'wd': 0.1},
}

DEBERTA_RESULT, DEBERTA_CFG = {}, {}
for tag in ('D1', 'D2'):
    cfg = DEBERTA_BEST[tag]
    print(f'{tag} {DATASET_NAMES[tag]:9s} DeBERTa  learning_rate={cfg["lr"]:g}  '
          f'batch_size={cfg["bs"]}  weight_decay={cfg["wd"]:g}')
    DEBERTA_CFG[tag] = dict(cfg, key=run_key(tag, 'DeBERTa', cfg))
    DEBERTA_RESULT[tag] = train_one(tag, 'DeBERTa', cfg)
```

4. Ensemble code

```
from scipy.stats import binomtest


def load_probs(key):
    z = np.load(PROBS_DIR / f'{key}.npz')
    return {k: z[k] for k in z.files}


def paired_test(y, pred_a, pred_b, n_boot=10000, seed=SPLIT_SEED):
    rng = np.random.default_rng(seed)
    wa, wb = pred_a != y, pred_b != y
    b = int((~wa & wb).sum())
```

```
    c = int((wa & ~wb).sum())
    p = binomtest(b, b + c, 0.5).pvalue if (b + c) else 1.0
    idx = rng.integers(0, len(y), size=(n_boot, len(y)))
    boot = wa[idx].mean(1) - wb[idx].mean(1)
    lo, hi = np.percentile(boot, [2.5, 97.5])
    return {'b': b, 'c': c, 'p': float(p),
            'diff_pp': float((wa.mean() - wb.mean()) * 100),
            'ci_lo_pp': float(lo * 100), 'ci_hi_pp': float(hi * 100)}


WEIGHTS = np.round(np.arange(0.0, 1.0001, 0.05), 2)
ENSEMBLE = {}

for tag in ('D1', 'D2'):
    pb = load_probs(BERT_CFG[tag]['key'])
    pdb = load_probs(DEBERTA_CFG[tag]['key'])
    assert np.array_equal(pb['val_labels'], pdb['val_labels'])
    assert np.array_equal(pb['test_labels'], pdb['test_labels'])
    yv, yt = pb['val_labels'], pb['test_labels']

    val_f1 = []
    for w in WEIGHTS:
        mix = w * pb['val_probs'] + (1 - w) * pdb['val_probs']
        val_f1.append(weighted_metrics(yv, mix.argmax(1))[3])
    best_w = float(WEIGHTS[int(np.argmax(val_f1))])

    ens_pred = (best_w * pb['test_probs'] + (1 - best_w) * pdb['test_probs']).argmax(1)
    acc, pre, rec, f1 = weighted_metrics(yt, ens_pred)

    members = {'BERT': pb['test_probs'].argmax(1),
               'DeBERTa': pdb['test_probs'].argmax(1)}
    mem_f1 = {k: weighted_metrics(yt, v)[3] for k, v in members.items()}
    stronger = max(mem_f1, key=mem_f1.get)
    pt = paired_test(yt, ens_pred, members[stronger])

    ENSEMBLE[tag] = {'weight_bert': best_w, 'weight_deberta': round(1 - best_w, 2),
                     'test_f1': round(f1, 4), 'test_accuracy': round(acc, 4),
                     'member_f1': {k: round(v, 4) for k, v in mem_f1.items()},
                     'stronger_member': stronger, 'paired': pt,
                     'degenerate': best_w in (0.0, 1.0),
                     'confusion': confusion_matrix(yt, ens_pred).tolist()}

    print(f'{tag} {DATASET_NAMES[tag]}')
    print(f'   weight_bert={best_w:.2f}  weight_deberta={1 - best_w:.2f}')
    print(f'   BERT {mem_f1["BERT"]:.4f}   DeBERTa {mem_f1["DeBERTa"]:.4f}   '
          f'ensemble {f1:.4f}')
    print(f'   McNemar p={pt["p"]:.4g}  error diff {pt["diff_pp"]:+.3f} pp  '
          f'95 pct CI [{pt["ci_lo_pp"]:+.3f}, {pt["ci_hi_pp"]:+.3f}]')
```